TÀI LIỆU HƯỚNG DẪN
QUY TRÌNH VÀ RÀNG BUỘC THỰC HIỆN DỰ ÁN TRÊN GIT/GITHUB

Tài liệu này mô tả chi tiết quy trình, các bước kỹ thuật và những ràng buộc bắt buộc dành cho sinh viên khi thực hiện làm việc nhóm trên kho lưu trữ mã nguồn Git/GitHub. Mục đích nhằm chuẩn hóa quy trình phát triển phần mềm, minh bạch hóa tiến độ làm việc và giúp giảng viên đánh giá chính xác mức độ đóng góp của từng cá nhân.

1. Thiết Lập Định Danh Cá Nhân (Bắt Buộc)

Mỗi sinh viên bắt buộc phải cấu hình thông tin cá nhân trên Git tại máy cục bộ (local machine) khớp với danh sách lớp. Hệ thống sẽ từ chối ghi nhận điểm đối với các tài khoản ẩn danh, tài khoản ảo hoặc thông tin không rõ ràng.

- Cú pháp cấu hình:
  git config --global user.name "MSSV_HoVaTen"
  git config --global user.email "email_sinh_vien@domain"

Ràng buộc nghiêm ngặt:

- Tuyệt đối không sử dụng máy hoặc tài khoản của sinh viên khác để commit thay.
- Mọi dấu hiệu commit hộ (một người push toàn bộ code cho cả nhóm) sẽ dẫn đến việc hủy kết quả của những thành viên không có lịch sử làm việc trên hệ thống.

2. Quản Lý Công Việc Bằng Issue (Task Board)

Mọi đầu việc từ nhỏ đến lớn (thiết kế CSDL, viết API, làm giao diện, fix bug) đều phải được khởi tạo thành Issue trên Git trước khi bắt tay vào viết code.

Quy định khi tạo Issue:

- Phân rõ trách nhiệm: Mỗi Issue chỉ gán (Assign) cho ĐÚNG 1 NGƯỜI chịu trách nhiệm chính.
- Gắn nhãn (Label) và Cột mốc (Milestone): Issue phải được gắn nhãn (vd: bug, enhancement, frontend, backend) và thuộc về một Milestone cụ thể (vd: Tuần 1, Tuần 2).

3. Chiến Lược Phân Nhánh (Branching Strategy)

Nhánh chính (main / master / develop) sẽ được nhóm trưởng/ giảng viên khóa lại (Protected). Mọi thao tác push mã nguồn trực tiếp lên nhánh chính đều bị hệ thống từ chối.

Quy tắc rẽ nhánh:

- Khi bắt đầu giải quyết một Issue, sinh viên phải tạo một nhánh cá nhân mới từ nhánh chính.
- Cú pháp đặt tên nhánh:
    - *feature/<MSSV>-issue-<Số_Issue>-<Tên_ngắn_gọn>*
    - Ví dụ: feature/23110001-issue-5-login-form

4. Tiêu Chuẩn Commit Mã Nguồn

Lịch sử commit là cơ sở để chấm điểm chuyên cần và quá trình phát triển mã nguồn.

Cấu trúc thông điệp (Commit Message):

- Tuân thủ Conventional Commits: <loại>: <mô tả ngắn gọn>
    - Các loại được chấp nhận: feat (tính năng mới), fix (sửa lỗi), refactor (tối ưu code), docs (tài liệu).
    - Ví dụ hợp lệ: "feat: thêm chức năng mã hóa mật khẩu", "fix: lỗi crash khi mất kết nối mạng".

Vệ sinh mã nguồn (Cực kỳ quan trọng):

- Bắt buộc sử dụng file .gitignore đúng chuẩn của ngôn ngữ/framework đang dùng.
- Nghiêm cấm commit các thư mục build (target, node_modules, bin), cấu hình cá nhân (.idea, .vscode), hoặc các file nhị phân lớn (.jar, .zip).

5. Quy Trình Tích Hợp (Merge Request / Pull Request)

Khi tính năng đã hoàn thiện trên nhánh cá nhân, sinh viên mở Merge Request (MR) để yêu cầu gộp code vào nhánh chính.

Ràng buộc khi Merge:

- Mô tả MR phải có nội dung chi tiết: Đã làm gì? Có ảnh hưởng đến module khác không? (Yêu cầu đính kèm ảnh chụp màn hình chứng minh code đã chạy thành công).
- Review chéo (Peer Review): Bắt buộc phải có ÍT NHẤT 1 THÀNH VIÊN KHÁC trong nhóm vào kiểm tra code, để lại nhận xét và bấm nút Approve.
- Nghiêm cấm hành vi tự tạo MR và tự Approve cho chính mình.

6. Cập Nhật Bảng Đóng Góp (README.md)

Tại thư mục gốc của repository, nhóm phải duy trì file README.md. Trong đó bắt buộc có một bảng ma trận thống kê sự đóng góp của các thành viên để đối chiếu khi chấm điểm bảo vệ cuối kỳ.

MSSV	Họ và Tên	Công việc đã thực hiện	Merge Requests

23110001	Nguyễn Văn A	Xây dựng core network (TCP/UDP)	!1, !3, !7
23110002	Trần Thị B	Thiết kế giao diện Client	!2, !4, !6

7. Chế Tài Xử Lý Vi Phạm

- Vi phạm quy tắc vệ sinh kho mã nguồn (Push file rác, file build): Trừ 20% tổng điểm nhóm.
- Phát hiện "Big Bang Commit" (Push dồn cục lượng lớn mã nguồn vào 1-2 ngày trước hạn nộp): Hủy kết quả điểm đánh giá quá trình chuyên cần.
- Gian lận đóng góp: Nếu khi vấn đáp cuối kỳ, sinh viên không giải thích được luồng logic của đoạn code do chính tài khoản mình commit -> Hủy toàn bộ điểm đề tài của cá nhân đó.